JavaScript Modulaire

Maîtriser les ES Modules pour des applications scalables et maintenables.

SUPINFO 1WEBD

Day 3 : Architecture & Organisation

AU PROGRAMME

01 Pourquoi la modularité ? (Contexte)

02 La syntaxe Import & Export

03 Configuration HTML (type="module")

04 Exports nommés vs Exports par défaut

05 Gestion des conflits et Namespaces

L'ÉVOLUTION DU JAVASCRIPT

> Du snippet à l'application SPA dans le navigateur.

AVANT

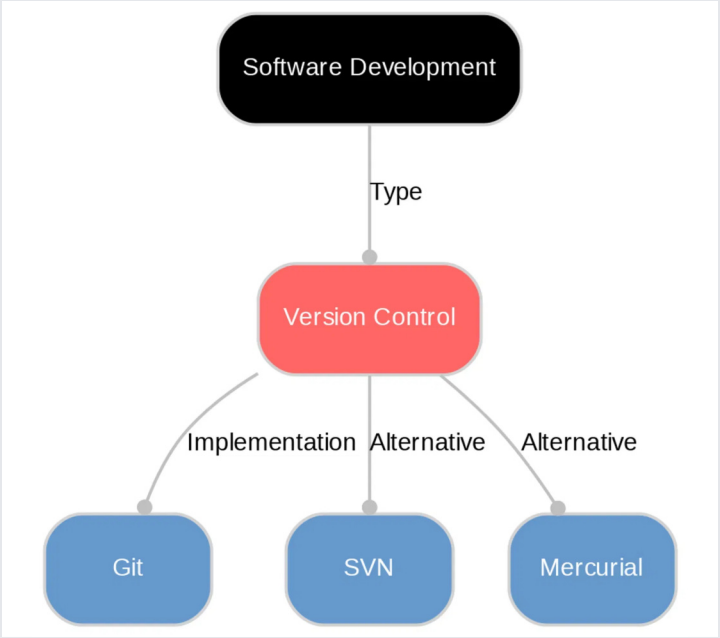
JS pour petites interactions, souvent désorganisé.

AUJOURD'HUI

Applications massives, code divisé en modules.

SOLUTION NATIVE

ES Modules, supportés par tous les navigateurs modernes.



Modularité pour une meilleure organisation.

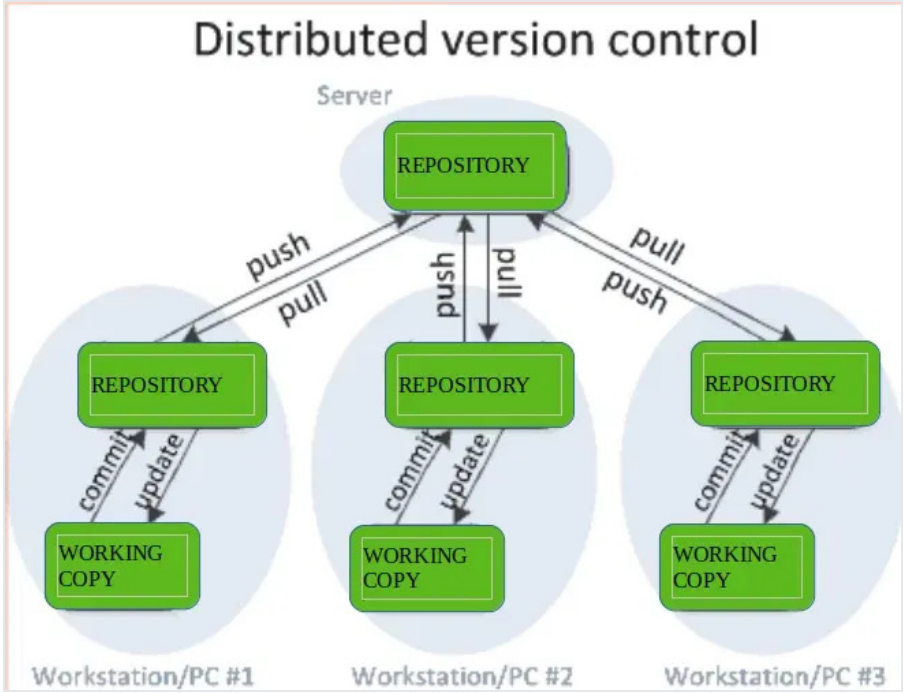
ACTIVER LES MODULES

> Pour utiliser `import` et `export`, vous devez indiquer au navigateur que votre script est un module.

SYNTAXE HTML

```
<script type="module" src="main.js"></script>
```

- **Chargement différé :** Les modules sont "defer" par défaut.
- **Portée isolée :** Pas de pollution de l'espace global (window).
- **Mode strict :** Les modules s'exécutent toujours en "use strict".



Les modules permettent une architecture distribuée et isolée.

PARTAGER SON CODE : EXPORT

L'instruction **export** permet de rendre des fonctions, variables ou classes disponibles pour d'autres fichiers.

01. DEVANT LA DÉCLARATION

```
export const maVar = 10;

export function direBonjour() {
  console.log("Hello !");
}
```

02. GROUPÉ EN FIN DE FICHIER

```
const fonc1 = () ⇒ { ... };
const fonc2 = () ⇒ { ... };

export { fonc1, fonc2 };
```

— *Seuls les éléments explicitement exportés sont visibles depuis l'extérieur du module.*

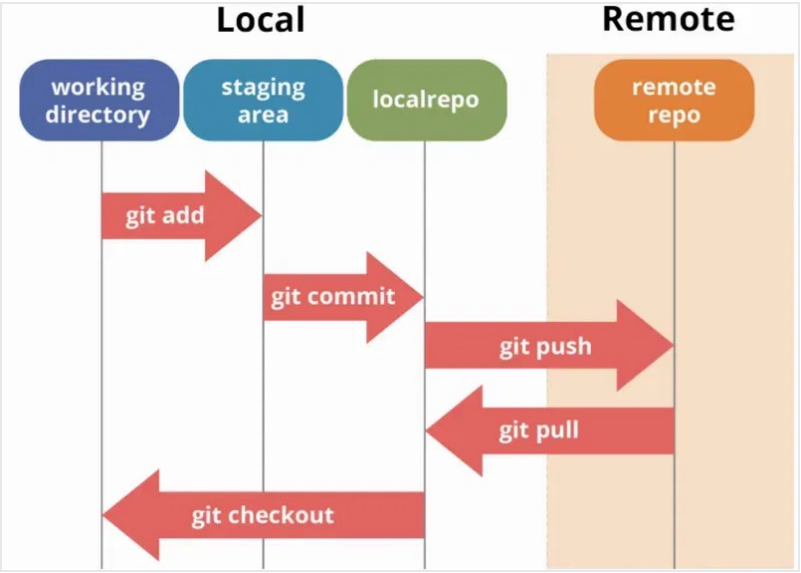
UTILISER UN MODULE : IMPORT

SYNTAXE DE BASE

```
import { element } from './chemin/module.js';
```

RÈGLES D'UTILISATION

- **Chemins :** Doit être relatif (./ ou ../) ou une URL absolue.
- **Extensions :** L'extension .js est obligatoire dans le navigateur.
- **Position :** Les imports sont statiques et doivent être en haut du fichier.



L'importation crée un flux de données entre vos modules.

EXPORT NOMMÉ VS PAR DÉFAUT

CARACTÉRISTIQUE	EXPORT NOMMÉ	EXPORT PAR DÉFAUT
Syntaxe Export	`export const A = ...`	`export default A`
Syntaxe Import	`import { A } from ...`	`import A from ...`
Quantité	Plusieurs par fichier	Un seul par fichier
Usage	Utilitaires, composants multiples	Composant principal, classe unique

Conseil : Utilisez les exports nommés pour la plupart de vos utilitaires afin de faciliter l'auto-complétion dans votre IDE.

ÉVITER LES COLLISIONS : L'ALIAS AS

LE PROBLÈME

Deux modules différents exportent une fonction avec le même nom (ex: `render()`). L'importation directe créerait un conflit dans votre fichier.

LA SOLUTION

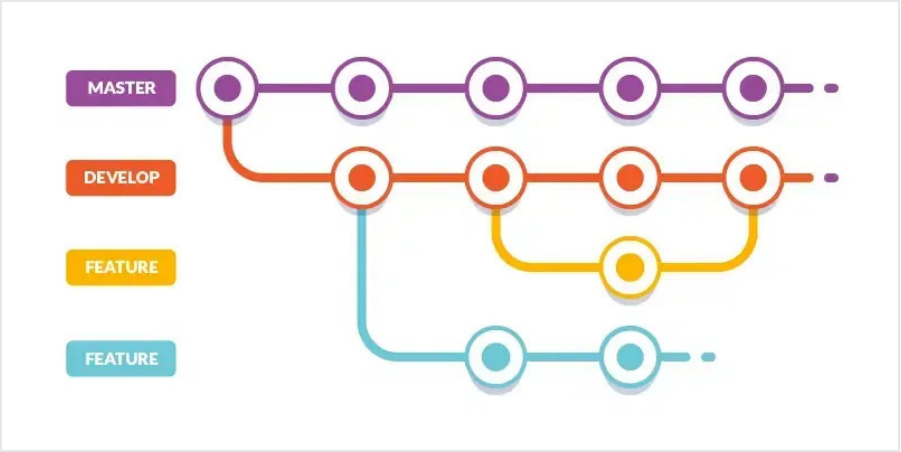
```
import { render as renderHome } from './home.js';
import { render as renderAbout } from './about.js';

// Utilisation sans conflit
renderHome();
renderAbout();
```

Le mot-clé **as** permet de renommer localement un élément importé, garantissant ainsi l'unicité des noms dans votre script.

ESPACES DE NOMS

> L'importation globale permet de regrouper tous les exports d'un fichier dans un seul objet "Namespace".



Structurez vos modules comme des branches organisées.

SYNTAXE & USAGE

```
import * as Utils from './utils.js';

// Accès via l'objet
Utils.maFonction();
console.log(Utils.maVariable);
```

- **Code propre :** Évite les listes d'imports interminables.
- **Organisation :** Structure les fonctionnalités par thématique.
- **Lisibilité :** On sait immédiatement d'où vient la fonction.

IMPORT MAPS

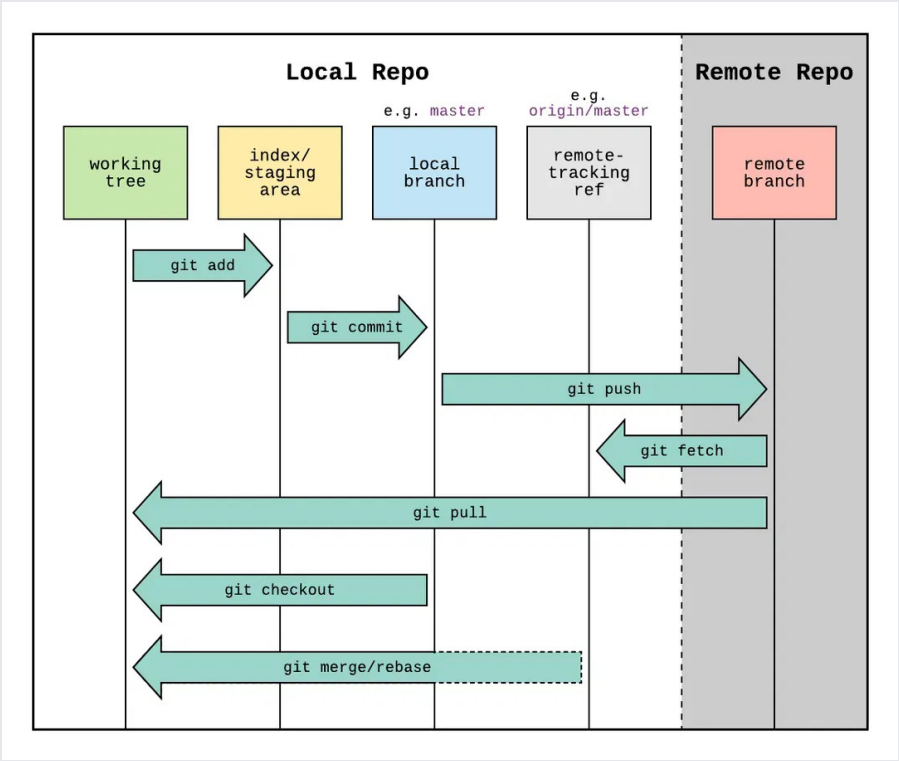
Les chemins relatifs complexes comme `../../services/auth.js` sont difficiles à maintenir. Les Import Maps simplifient cela via des alias.

CONFIGURATION HTML

```
<script type="importmap">
{
  "imports": {
    "auth": "./services/auth.js"
  }
}
</script>
```

UTILISATION DANS LE JS

```
import { login } from 'auth';
```



Les Import Maps résolvent les chemins de manière centralisée.

CONCLUSION : POINTS CLÉS

1 **Indispensable Modularité**

La modularité est le pilier du développement moderne pour créer des applications scalables et maintenables.

2 **Import & Export**

Utilisez **export** pour partager vos fonctionnalités et **import** pour les consommer là où elles sont nécessaires.

3 **Organisation & Performance**

Privilégiez les modules natifs (ESM) et organisez vos fichiers par responsabilité unique (un fichier = une mission).

"Un code bien organisé est un code qui **évolue sans douleur**."

GitHub

Questions ?

Merci de votre attention !

RESSOURCES

MDN Web Docs : ES Modules
developer.mozilla.org

JavaScript.info : Modules
javascript.info/modules

V8.dev : Modules
v8.dev/features/modules